hashlock.

Security Audit

Your Wallet (Crypto Wallet)

November 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

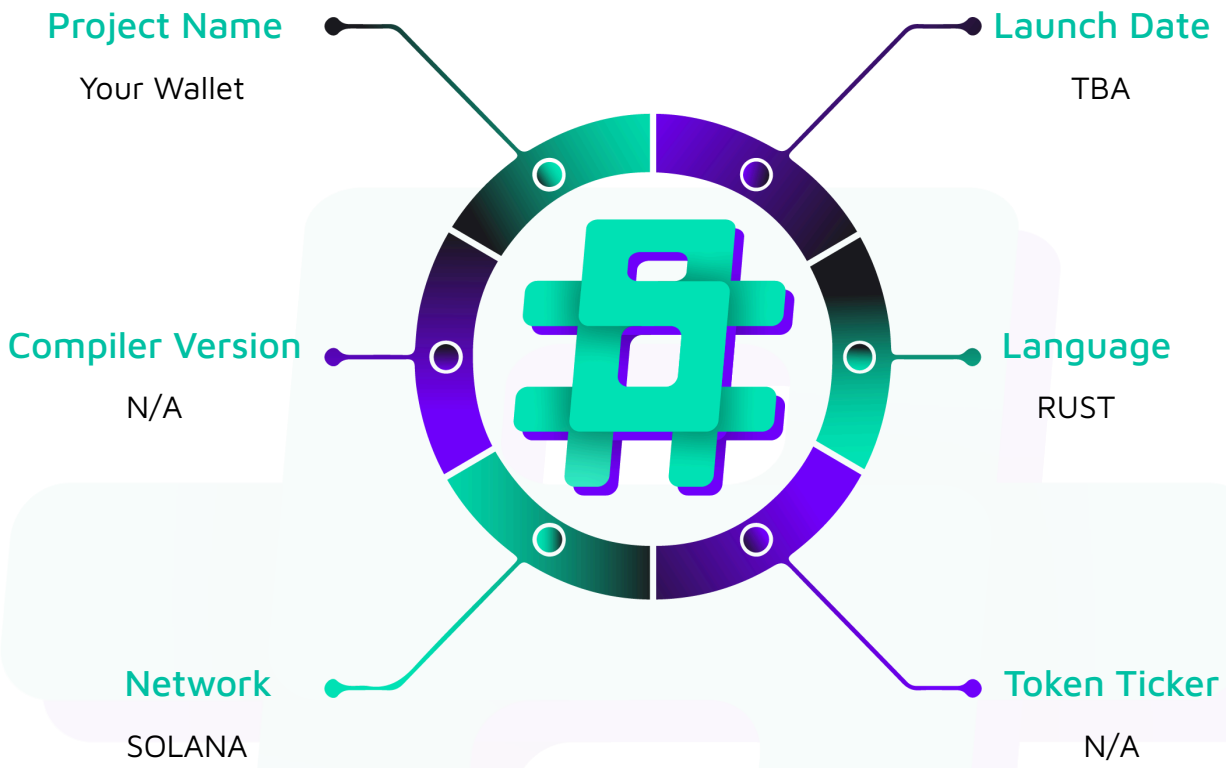
The Your Wallet team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

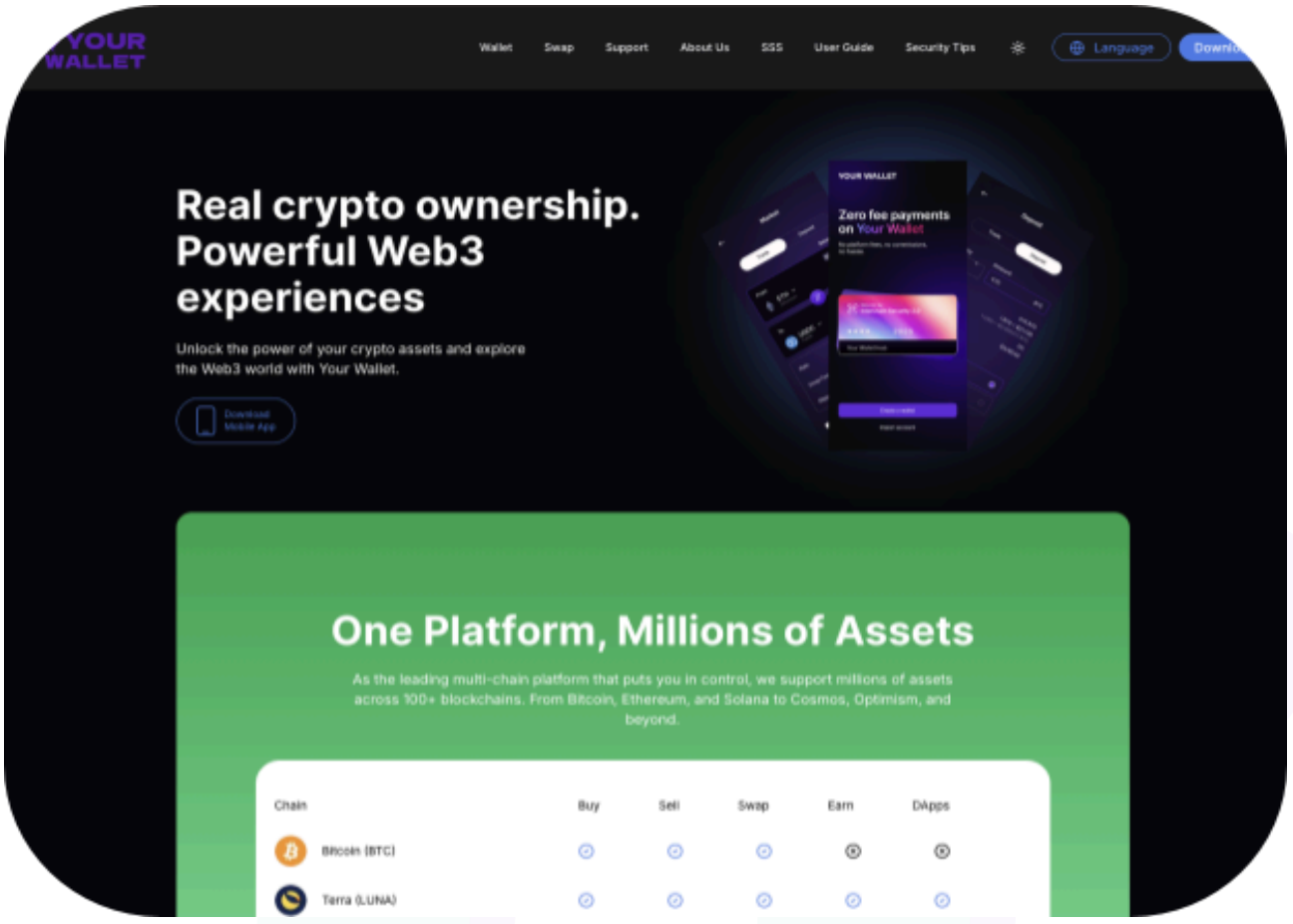
Your Wallet is a comprehensive mobile and desktop Web3 wallet designed to provide users with full control over their digital assets. It enables seamless buying, selling, swapping, and staking of cryptocurrencies. The platform emphasizes user privacy and security, offering features like encrypted backups, biometric authentication, and proactive alerts for risky transactions. Additionally, Your Wallet facilitates easy deposits from major exchanges and supports decentralized applications (dApps), making it a versatile tool for both crypto enthusiasts and developers seeking to engage with the Web3 ecosystem.

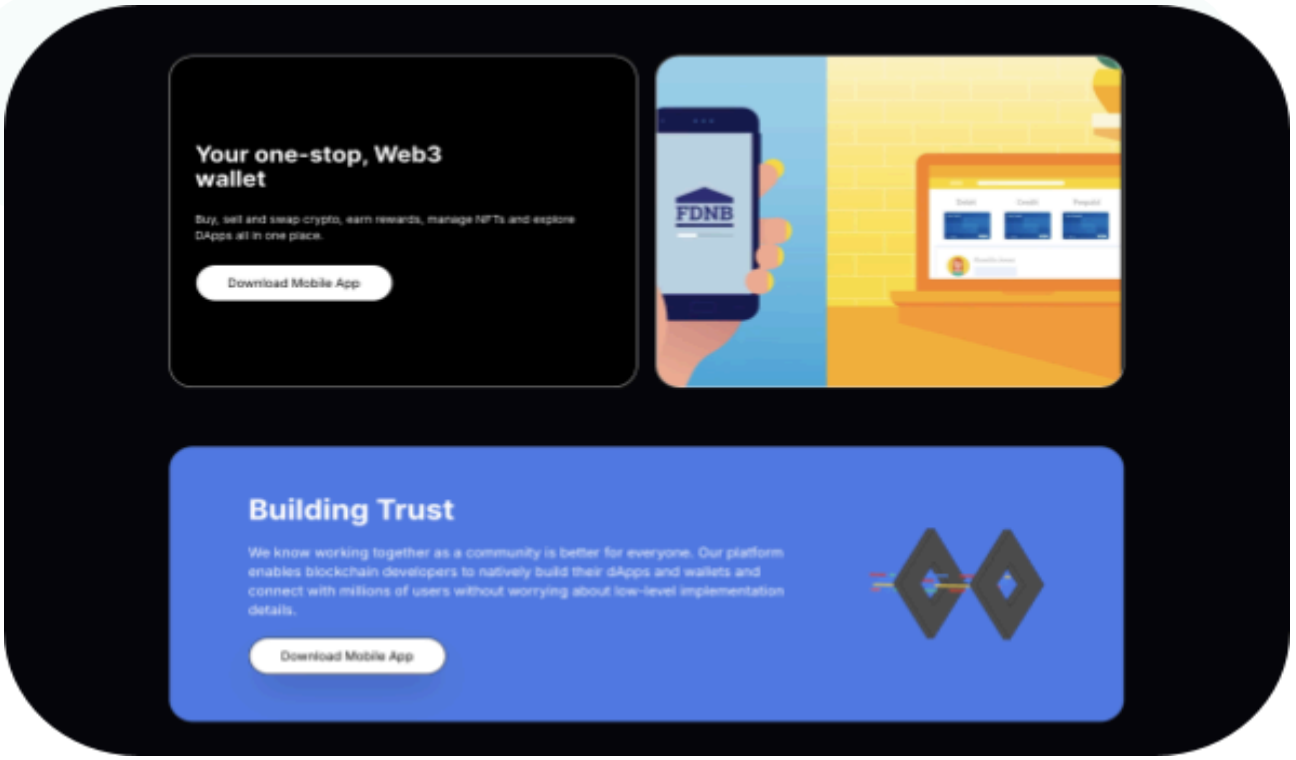
Project Name: Your Wallet
Project Type: Crypto Wallet
Website: https://www.yourwallet.tr/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Your Wallet project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Your Wallet Staking Programs
Platform	Rust/Solana
Audit Date	November, 2025
Program 1	src/processor/admin.rs
Program 2	src/processor/close.rs
Program 3	src/processor/helpers.rs
Program 4	src/processor/initialize.rs
Program 5	src/processor/mod.rs
Program 6	src/processor/rewards.rs
Program 7	src/processor/stake.rs
Program 8	src/assertions.rs
Program 9	src/entrypoint.rs
Program 10	src/error.rs
Program 11	src/instruction.rs
Program 12	src/state.rs
Program 13	src/utils.rs
Audited GitHub Commit Hash	ce155f27e4960ddd3665e3edc80ddccde6b62c9b
Fix Review GitHub Commit Hash	3caaa4ce76684eaa1a00e26d488f7188e3b0fc8b

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High-severity vulnerabilities

3 Medium-severity vulnerabilities

2 Low-severity vulnerabilities

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
src/lib.rs - Declares the program's modules (assertions , entrypoint , error , instruction , processor , state , utils). - Specifies the program's unique on-chain ID.	**Program achieves this functionality.**
src/entrypoint.rs - Provides the main entry point for the program, which is called by the Solana runtime. - Delegates all instruction processing to the processor module.	**Program achieves this functionality.**
src/instruction.rs - Defines the StakePoolInstruction enum, which enumerates all possible actions the program can perform. - Each variant corresponds to a specific operation, such as initializing the pool, staking, unstaking, and claiming rewards.	**Program achieves this functionality.**
src/error.rs - Defines custom error types that provide specific reasons for transaction failures. - Enables more precise error handling and debugging.	**Program achieves this functionality.**
src/state.rs - Defines the on-chain data structures, StakePool and StakeAccount, which store the program's	**Program achieves this functionality.**

state. - Includes methods for data serialization/deserialization and reward calculation logic.	
src/assertions.rs - Contains a suite of validation functions to ensure the integrity of accounts and inputs. - These functions check for correct ownership, PDA derivations, and other security-critical properties.	**Program achieves this functionality.**
src/utils.rs - Provides a collection of helper functions for common on-chain operations. - Includes utilities for creating new accounts, transferring tokens securely (with fee support), and closing accounts to recover rent.	**Program achieves this functionality.**
src/processor/mod.rs - Acts as the central instruction router for the program. - It deserializes incoming instruction data and dispatches it to the appropriate handler function.	**Program achieves this functionality.**
src/processor/initialize.rs - Handles the InitializePool instruction to create and configure a new stake pool. - Sets the initial parameters, such as reward rate, lockup period, and mints.	**Program achieves this functionality.**

src/processor/stake.rs - Manages the core staking and unstaking logic. - The stake function creates a user-specific StakeAccount and transfers tokens into the pool, while the unstake function allows users to withdraw their funds.	**Program achieves this functionality.**
src/processor/rewards.rs - Governs the reward distribution and funding mechanisms. - The claim_rewards function allows users to collect their earned rewards, and fund_rewards allows anyone to deposit additional reward tokens into the pool.	**Program achieves this functionality.**
src/processor/admin.rs - Implements administrative functions for managing the stake pool. - Includes logic for updating pool settings and a secure, two-step process for transferring administrative authority.	**Program achieves this functionality.**
src/processor/close.rs - Handles the CloseStakeAccount instruction, allowing users to close their stake account. - This function ensures that the account is empty before closing it and returning the rent to the user.	**Program achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Your Wallet project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Your Wallet project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] initialize_pool - Missing Token Account Ownership Validation

Description

The protocol NEVER validates that the `stake_vault` and `reward_vault` token accounts are actually owned by the pool PDA, which means an attacker can pass any account that they own as the vault.

Vulnerability Details

During `initialize_pool`, anyone can pass any arbitrary token account as the vault, and the pool will store these addresses without verifying ownership. An attacker can pass token accounts they control with no way to change these parameters as well.

The `verify_token_account` function only checks if the vault account holds the correct type of token (the mint), not who owns the `vault`.

```
pub fn verify_token_account(
    token_account: &AccountInfo,
    expected_mint: &Pubkey,
) -> Result<(), ProgramError> {
    let account_data = token_account.try_borrow_data()?;
    let account = StateWithExtensions::<TokenAccount>::unpack(&account_data)?;
    if &account.base.mint != expected_mint {
        return Err(StakePoolError::InvalidMint.into());
    }
    // @Missing: if &account.base.owner != expected_owner
    Ok(())
```

```
}
```

Impact

If the attacker passed the `stake_vault` address as the one that they control, then all the staked tokens will go to the attacker, resulting in a complete loss of funds

Recommendation

Update `initializa_pool` to validate vault ownership

Status

Resolved

[H-02] state#calculate_rewards - Unsafe reward calculation allows for Reward Vault Drain

Description

The protocol fails to enforce a minimum `lockup_period`, allowing an admin to set it to a trivially short duration, such as one second. Since rewards are not prorated based on staking duration, this allows any user to stake funds, wait for the negligible lockup to pass, and immediately claim a full, unearned reward, enabling a rapid drain of the reward vault.

Vulnerability Details

In the `initialize::initialize_pool` function, the lockup_period is only validated to be non-zero, and no minimum timelock has been enforced:

```
if lockup_period < 0 {

    msg!("Lockup period cannot be negative: {}", lockup_period);

    return Err(StakePoolError::InvalidParameters.into());

}
```

Second, the `calculate_rewards` function in `program/src/state.rs` does not factor the `time_staked` into the reward amount. It only checks if `time_staked` has surpassed the `lockup_period`. If it has, the full reward is granted based on the `reward_rate`, not the duration.

```
pub fn calculate_rewards(

    &self,

    amount_staked: u64,

    stake_timestamp: i64,

    current_time: i64,

) -> Result<u64, ProgramError> {

    let time_staked = current_time
```

```
        .checked_sub(stake_timestamp)
        .ok_or(StakePoolError::NumericalOverflow)?;
    if time_staked < self.lockup_period {
        return Ok(0);
    }
    const SCALE: u128 = 1_000_000_000;
    let rewards = (amount_staked as u128)
        .checked_mul(self.reward_rate as u128)
        .ok_or(StakePoolError::NumericalOverflow)?
        .checked_div(SCALE)
        .ok_or(StakePoolError::NumericalOverflow)? as u64;
    Ok(rewards)
}
```

As per the result of this implementation, an attacker can create a new stake pool with a high `reward_rate` and a `lockup_period` of just 1 second with a massive amount of tokens. Then, the attacker calls `unstake` or `claim_rewards`. The `calculate_rewards` function sees that `time_staked` is greater than `lockup_period` and pays out the entire, massive reward.

Impact

Complete loss of all rewards for legitimate stakers, which also leads to pool-level protocol insolvency as the reward vault is `drained`.

Recommendation

The reward calculation should be proportional to the time staked, or a minimum lockup period should be enforced.

Status

Resolved

Medium

[M-01] utils.rs#create_account - PDA Account Creation Vulnerable to Denial of Service

Description

The `create_account` function in `utils.rs` and its usage in `initialize.rs` (line 103) and `stake.rs` (line 197) are vulnerable to front-running Denial of Service (DoS) attacks. An attacker can calculate the PDA address in advance and send rent-exempt lamports to that address before the legitimate transaction is processed, causing the account creation to fail.

Vulnerability Details

The `create_account` function uses `solana_program::system_instruction::create_account`, which is designed to create a new account. A key characteristic of this instruction is that it will fail if the target account address (in this case, a PDA) already holds a non-zero lamport balance.

Because PDA addresses are deterministic, an attacker can compute the address for a future `stake_pool` or `StakeAccount` before it is created. By sending a small, rent-exempt amount of SOL to this address, the attacker "claims" it. When the legitimate user's transaction attempts to create the account at that same address, the `create_account` instruction fails, as the account is no longer "new." This effectively blocks the creation of the pool or stake account.

```
pub fn create_account<'a>(
    target_account: &AccountInfo<'a>,
    funding_account: &AccountInfo<'a>,
    _system_program_account: &AccountInfo<'a>,
    size: usize,
    owner: &Pubkey,
    signer_seeds: Option<&[&[&[u8]]]>,
```

```
) -> ProgramResult {
    let rent = Rent::get()?;
    let lamports: u64 = rent.minimum_balance(size);
    let create_account_ix = solana_program::system_instruction::create_account(
        funding_account.key,
        target_account.key,
        lamports,
        size as u64,
        owner,
    );
    // This will fail if target_account already has lamports
    invoke_signed(
        &create_account_ix,
        &[funding_account.clone(), target_account.clone()],
        signer_seeds.unwrap_or(&[]),
    )
}
```

The used PDA seeds are:

```
vec![
    b"stake_pool".to_vec(),
    authority.as_ref().to_vec(),
    stake_mint.as_ref().to_vec(),
]
```

Impact

- Pool creation can be permanently blocked for specific authority/mint combinations.

- Legitimate users cannot create their intended stake pools.
- Similar DoS can occur for stake account creation (multiple accounts per user).
- Attacker cost is minimal (rent-exempt amount is recoverable)

Recommendation

Replace `create_account` with an alternative approach that checks if the account exists and handles pre-funded accounts.

Status

Resolved

[M-02] helpers.rs#verify_token_account - Token-2022 Extensions Can Break Protocol Invariants

Description

The protocol supports Token-2022 mints but doesn't validate or restrict potentially dangerous extensions. Several Token-2022 extensions can break protocol assumptions

Vulnerability Details

The core of the vulnerability lies in the `verify_token_account` function, which is responsible for verifying the token accounts for new stake pools. This function accepts `token_account` and `expected_mint` accounts for both `stake_mint` and `reward_mint`, which can be standard SPL tokens or Token-2022 tokens. However, it fails to check if the extensions are enabled on these mints. Token-2022 introduces powerful but risky extensions that can undermine the protocol's security guarantees if not explicitly handled.

Since the protocol does not validate these extensions, it is vulnerable to any pool created with a malicious Token-2022 mint.

```
pub fn verify_token_account(

    token_account: &AccountInfo,

    expected_mint: &Pubkey,

) -> Result<(), ProgramError> {

    let account_data = token_account.try_borrow_data()?;

    let account = StateWithExtensions::<TokenAccount>::unpack(&account_data)

        .map_err(|_| StakePoolError::InvalidTokenProgram)?;

    if &account.base.mint != expected_mint {

        return Err(StakePoolError::InvalidMint.into());

    }

    // <@ No extension validation

    Ok(())
```

```
}
```

Impact

Certain Problematic extensions can lead to unintended behavior in the protocol:

1. Mint authority can close the mint account, rendering all stakes worthless
2. Custom hook logic on transfers can:
    - Block transfers entirely
    - Redirect tokens to different accounts
    - Charge unexpected fees
    - Cause reentrancy issues
3. Permanent Delegate: Can forcibly transfer tokens from the vault
4. Freeze Authority: Can freeze token accounts, preventing unstaking

Recommendation

Add `extension` validation in either `initialize_pool` or `verify_token_account`

Status

Resolved

[M-03] initialize.rs#initialize_pool - Mint Freeze Authority Can Lock User Funds

Description

The `initialize_pool` function does not check if the `stake_mint` has a freeze authority. This allows a malicious actor to create a pool with a freezable token, enabling them to lock user funds permanently.

Vulnerability Details

The `freeze_authority` in an SPL token mint is a feature that allows a designated account to freeze any token account holding that token, preventing all transfers. The `initialize_pool` function fails to verify whether this authority is set on the `stake_mint`. If a pool is created with a mint that has a freeze authority, the authority holder can unilaterally and permanently freeze the stake accounts of any user who deposits into the pool, rendering their funds inaccessible.

```
if stake_mint.decimals != reward_mint.decimals {

    msg!("Mismatch token decimals");

    return Err(StakePoolError::MismatchTokenDecimals.into());

}

// No check for freeze authority on stake_mint
```

Impact

Permanent loss of user funds in a particular pool only if the pool's mint owner decides to freeze the token transfers

Recommendation

Add a check in `initialize_pool` to ensure that the `stake_mint` does not have a freeze authority.

Status

Resolved

Low

[L-01] admin.rs - Centralized Control over Reward rate

Description

The pool authority can unilaterally change the `reward_rate` at any time using the `update_pool` function. This centralization of power allows the authority to alter reward rates post-stake, which can be unfair to users who staked expecting a different rate.

Impact

This will result in users getting a lower than expected reward than they have staked.

Recommendation

To mitigate this centralization risk, consider implementing a time-lock mechanism for reward rate changes. This would introduce a delay between the announcement of a rate change and its execution, giving users time to react. Or implement a mechanism where the new reward rate doesn't impact old stakes.

Status

Resolved

[L-02] Programs - Potential Future Account Size Issues

Description

The `StakePool::save()` method doesn't validate that the serialized data fits within the allocated account size. While the current size calculation is correct (278 bytes), if new fields are added in future versions, this `could` cause silent data truncation at the time of protocol upgrade

Recommendation

A defensive check should be used to ensure that the account data size is not too small for serialized data

Status

Resolved

QA

[Q-01] initialize.rs - Permissionless Pool Creation

Description

The `initialize_pool` instruction is permissionless - any user can create a stake pool with any token mint and set themselves as the authority. While this is a feature (permissionless staking), it can lead to:

Spam or Scam pools with attractive parameters but no actual rewards. Confusion for users trying to find legitimate pools

Recommendation

Consider implementing some sort of permission system to avoid spamming.

Status

Resolved

[Q-02] state.rs#calculate_rewards - Unsafe Casting

Description

The function uses an unsafe cast from `u128` to `u64`:

```
let rewards = (amount_staked as u128)
    .checked_mul(self.reward_rate as u128)
    .ok_or(StakePoolError::NumericalOverflow)?
    .checked_div(SCALE)
    .ok_or(StakePoolError::NumericalOverflow)? as u64;
```

Recommendation

While the math makes overflow unlikely, this should use `try_into()` for safety.

Status

Acknowledged

Centralisation

The Your Wallet project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Your Wallet project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au